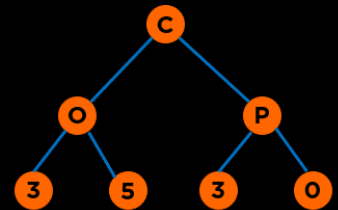


Exam 1 Review - A



Categories of Data Structures

Linear Ordered

Lists

Stacks

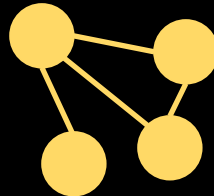
Queues



Non-linear Ordered

Trees

Heaps



Agenda

- **Record this Lecture**
- **Announcements**
- **Exam 1 Logistics**
- **Exam Review A: Module 2 and 3**

Exam 1 Logistics

- Date and timings :
 - For campus students (excluding students who need accommodations): Exam 1 will be on Oct 6 (Tuesday) at 8:20 pm to 10:10 pm on campus in-person. The location of the exam is dependent on your last name.
 - PUGH Pugh Hall 170 : Abichou - Kalariya
 - TUR Turlington Hall L005 : Kanige - Perojevic
 - WEIL Weil Hall 0270: Petrychenko - Zozaya
 - For campus students who need accommodations as per DRC: Exam 1 will be on Oct 6 (Tuesday). You can take the exam on Honorlock anytime at home between 6 pm to 11:59 pm at your home.
- The exam duration is 1 hr 50 minutes hours.
- You are allowed the following for your exam:
 - one double sided handwritten sheet of notes (letter size/A4 page).
 - 4 double sided blank sheets of scratch paper (letter size/A4 page).

Exam 1 Topics and Expectations

https://docs.google.com/document/d/1mfFdECreuzJLSRuCWvGjSMqbiQ88ZvFY_9UHE3I9iNM/edit?usp=sharing

Mini Review – Complexity

What is the computational complexity of adding an item to a stack in the worst case in terms of Big O notation?

Mini Review – Complexity

What is the computational complexity of adding an item to a stack in the worst case in terms of Big O notation?

Answer: $O(1)$

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
int k;  
for (int i=1; i < n; i++)  
    for (int j=n; j>1; j=j/2)  
        k = i*j;
```

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
int k;  
for (int i=1; i < n; i++)  
    for (int j=n; j>1; j=j/2)  
        k = i*j;
```

Answer: $O(n \log_2 n)$

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
int i = 1;
while (i < n)
{
    i = i * m;
}
```

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
int i = 1;
while (i < n)
{
    i = i * m;
}
```

Answer: $O(\log_m n)$

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
result = 0
for (int i = 0; i < n; i++)
    result += i;
for (int j = 1; j < m; j *= 2)
    result *= j;
```

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
result = 0
for (int i = 0; i < n; i++)
    result += i;
for (int j = 1; j < m; j *= 2)
    result *= j;
```

Answer: $O(n + \log_2 m)$

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
for (int i=n; i>0; i/=2)
    for (int j=1; j<i; j++)
        sum = sum + 1;
```

Mini Review – Complexity

What is the computational complexity of the following code snippet?

```
for (int i=n; i>0; i/=2)
    for (int j=1; j<i; j++)
        sum = sum + 1;
```

Answer: $O(n)$

Mini Review – Complexity

Which code snippet will take less time to execute on a computer?

A

```
for (int i=1; i < n; i++)  
    for (int j=1; j < n; j++)  
        sum = sum + 1;
```

B

```
for (int i=1; i < 2n; i++)  
    for (int j=1; j < 2n; j++)  
        sum = sum + 1;
```

Mini Review – Complexity

Which code snippet will take less time to execute on a computer?

A

```
for (int i=1; i < n; i++)  
    for (int j=1; j < n; j++)  
        sum = sum + 1;
```

B

```
for (int i=1; i < 2n; i++)  
    for (int j=1; j < 2n; j++)  
        sum = sum + 1;
```

Answer: A

Mini Review – Complexity

Which code snippet will have a higher growth rate asymptotically in terms of Big O notation?

A

```
for (int i=1; i < n; i++)  
    for (int j=1; j < n; j++)  
        sum = sum + 1;
```

B

```
for (int i=1; i < 2n; i++)  
    for (int j=1; j < 2n; j++)  
        sum = sum + 1;
```

Mini Review – Complexity

Which code snippet will have a higher growth rate asymptotically in terms of Big O notation?

A

```
for (int i=1; i < n; i++)  
    for (int j=1; j < n; j++)  
        sum = sum + 1;
```

B

```
for (int i=1; i < 2n; i++)  
    for (int j=1; j < 2n; j++)  
        sum = sum + 1;
```

Answer: A and B grow at the same rate asymptotically

Mini Review – Linked Lists

Consider a class `List` that implements an `ordered` list backed by a singly linked list with a head pointer. The invariant “`ordered`” is maintained always. Given that representation, what is the worst-case time complexity of the following operations? Assume the list is sorted in ascending order.

- A. Insert an item
- B. Finding the minimum element
- C. Delete the largest element from list
- D. Finding the largest element
- E. Finding a random element, `n`
- F. Deleting the minimum element in the list

Mini Review – Linked Lists

Consider a class List that implements an **ordered** list backed by a singly linked list with a head pointer. The invariant “**ordered**” is maintained always. Given that representation, what is the worst-case time complexity of the following operations? Assume the list is sorted in ascending order.

- A. Insert an item : $O(n)$
- B. Finding the minimum element : $O(1)$
- C. Delete the largest element from list : $O(n)$
- D. Finding the largest element : $O(n)$
- E. Finding a random element, n : $O(n)$
- F. Deleting the minimum element in the list : $O(1)$

Mini Review – Stacks

Postfix Evaluation “2 3 1 * + 9 -“. We scan all elements one by one.

Mini Review – Stacks

Postfix Evaluation “2 3 1 * + 9 -“. We scan all elements one by one.

- 1) Scan ‘2’, it’s a number, so push it to stack. Stack contains ‘2’
- 2) Scan ‘3’, again a number, push it to stack, stack now contains ‘2 3’ (from bottom to top)
- 3) Scan ‘1’, again a number, push it to stack, stack now contains ‘2 3 1’
- 4) Scan ‘*’, it’s an operator, pop two operands from stack, apply the * operator on operands, we get $3*1$ which results in 3. We push the result ‘3’ to stack. Stack now becomes ‘2 3’.
- 5) Scan ‘+’, it’s an operator, pop two operands from stack, apply the + operator on operands, we get $3 + 2$ which results in 5. We push the result ‘5’ to stack. Stack now becomes ‘5’.
- 6) Scan ‘9’, it’s a number, we push it to the stack. Stack now becomes ‘5 9’.
- 7) Scan ‘-’, it’s an operator, pop two operands from stack, apply the – operator on operands, we get $5 - 9$ which results in -4. We push the result ‘-4’ to stack. Stack now becomes ‘-4’.
- 8) There are no more elements to scan, we return the top element from stack (which is the only element left in stack).

Output Prediction / Coding Questions

<https://onlinegdb.com/BJ4hyD7vP>

<https://onlinegdb.com/BJ6gewQDP>

<https://onlinegdb.com/OefiHIntV>

Mini Review – Coding questions

- Check whether a string is a Palindrome using a Stack.
- Write C++ code for adding an element in the rear in a doubly linked list consisting of a head and tail.
- Write C++ code to find the median in a single linked using slow and fast pointer technique. –
Edugator 4.5

Mini Review – Coding questions

- Design a Stack data structure in C++ that supports push, pop and min operations in $O(1)$ time. You can use an STL Tack if needed.

Mini Review – Coding questions

- Design a Stack data structure in C++ that supports push, pop and min operations in $O(1)$ time. You can use an STL Tack if needed.

```
class MinStack {
private:
    stack<int> s;          // main stack
    stack<int> minStack; // keeps track of current minimums

public:
    // Push an element onto the stack
    void push(int x) {
        s.push(x);

        // If minStack is empty or x <= current min, push to minStack
        if (minStack.empty() || x <= minStack.top())
            minStack.push(x);
        else
            minStack.push(minStack.top());
    }
}
```

Mini Review – Coding questions

- Design a Stack data structure in C++ that supports push, pop and min operations in $O(1)$ time. You can use an STL Tack if needed.

```
// Pop the top element
void pop() {
    if (s.empty())
        throw underflow_error("Stack is empty");
    s.pop();
    minStack.pop();
}
```

```
// Get the current minimum in  $O(1)$ 
int min() {
    if (minStack.empty()) {
        throw underflow_error("Stack is empty");
    }
    return minStack.top();
}
```

```
};
```

Questions